

Excellent question. The diagram contains **multiple flows at once**, which can be confusing initially. The easiest way is to **follow one segment at a time** from top to bottom.

Let's map the flow exactly as the Linux kernel processes it.

Step 0: Application Calls `send()`



What happens?

- Your application enters the kernel.
- The application waits.
- The kernel now owns the data.

Step 1: Transport Layer (TCP)

Suppose your application sends **5000 bytes**.

TCP cannot send all 5000 bytes in one segment (assuming $MSS \approx 1460$ bytes).

So the kernel performs segmentation.



Segment 3 1460 Bytes

Segment 4 620 Bytes

Each segment gets its own **TCP Header**.

Segment 1

```
+-----+
| TCP Header          |
+-----+
| 1460 Bytes Data     |
+-----+
```

Segment 2

```
+-----+
| TCP Header          |
+-----+
| 1460 Bytes Data     |
+-----+
```

Important: TCP does **not** send all four segments as one packet. Each segment is processed independently by the lower layers.

Step 2: Network Layer (Executed for EVERY Segment)

Now consider **Segment 1**.

TCP Segment 1



Network Layer

The kernel performs:

Check Routing Table

Destination IP = 192.168.1.20

↓

Which Interface?

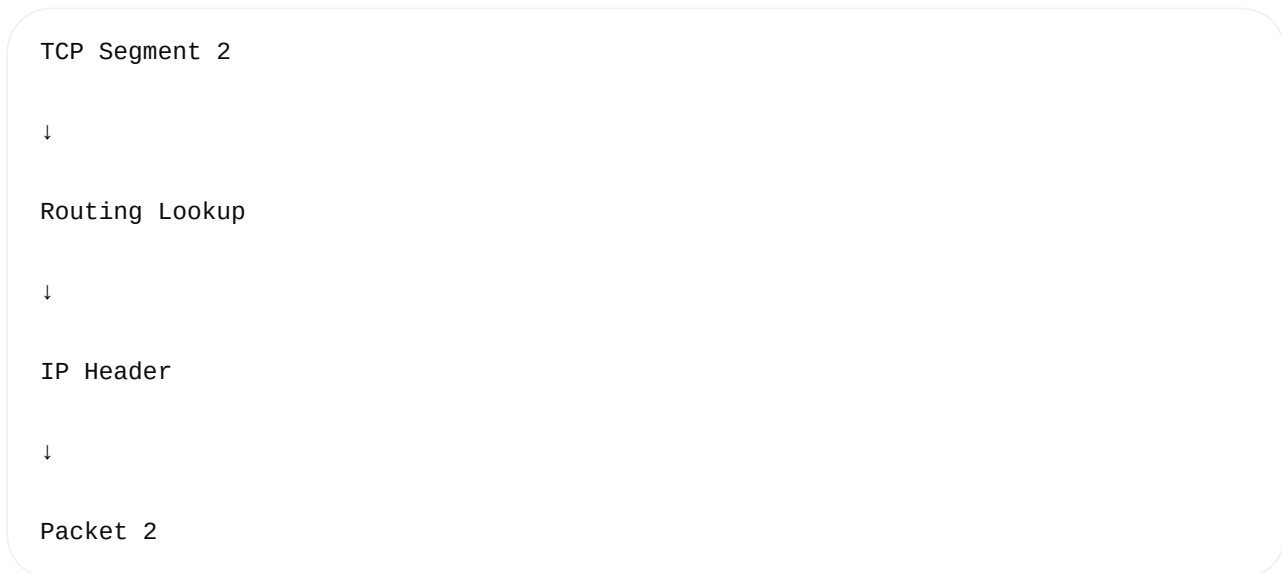
eth0

Then it adds an IP header.

```
+-----+
| IP Header           |
```



Now repeat the exact same process for Segment 2.



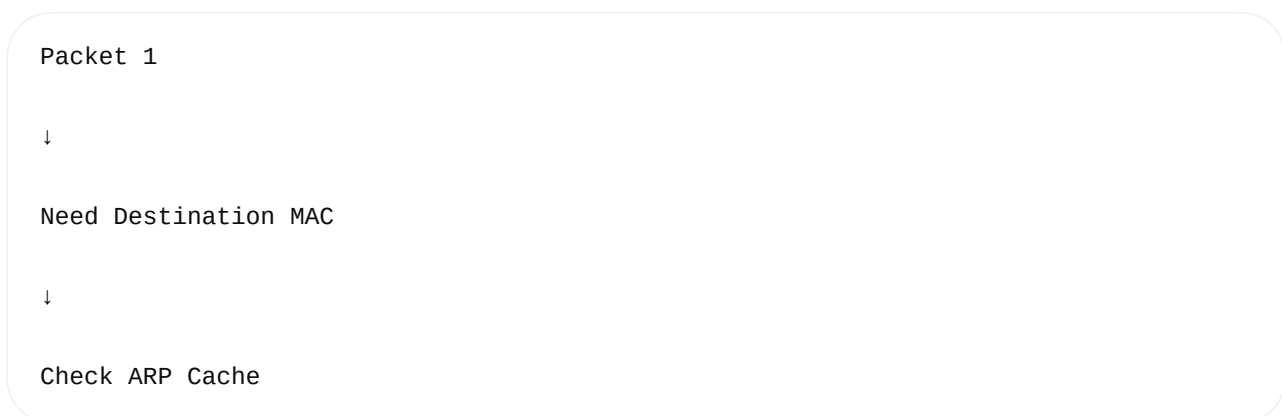
The same happens for Segments 3 and 4.

Routing is performed for every packet, although Linux may cache routing information to avoid repeating expensive lookups.

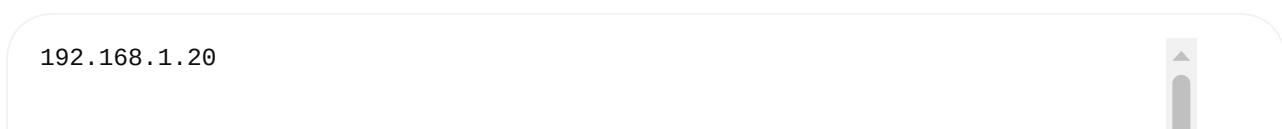
Step 3: Data Link Layer (Executed for EVERY Packet)

Now take **Packet 1**.

The kernel needs the destination MAC address.



If the ARP cache already contains:



↓

BB:BB:BB:BB

the kernel immediately builds the Ethernet frame.

```
+-----+
| Ethernet Header |
+-----+
| IP Packet       |
+-----+
| CRC             |
+-----+
```

Now the kernel processes Packet 2.

Again:

Need Destination MAC

↓

Check ARP Cache

Since the ARP entry already exists, it simply reuses it.

The same happens for Packet 3 and Packet 4.

ARP is not broadcast for every packet. The ARP request happens only if there is no valid cache entry (or it has expired). All packets then reuse the cached MAC address until the entry times out.

Step 4: Physical Layer

Each Ethernet frame is transmitted individually.

Frame 1

↓

Bits

↓

Cable

↓

Switch

Frame 2

↓

Bits

↓

Cable

↓

Switch

Step 5: Switch Processing

Frame 1 arrives.

Frame 1

↓

Switch

↓

Read Source MAC

AA-AA-AA

↓

Learn

AA-AA-AA → Port1

Then the switch reads:

Destination MAC

BB-BB-BB

It searches its MAC table.

MAC Table

BB-BB-BB

↓

Port2

The frame is forwarded only to Port 2.

Now Frame 2 arrives.

The switch repeats:

Learn Source MAC

↓

Lookup Destination MAC

↓

Forward

This happens for every frame.

The switch processes **each Ethernet frame independently**.

Step 6: PC-B Receives Each Frame

Frame 1

Bits

↓

Ethernet Frame

↓

Remove Ethernet Header

↓

IP Packet

↓

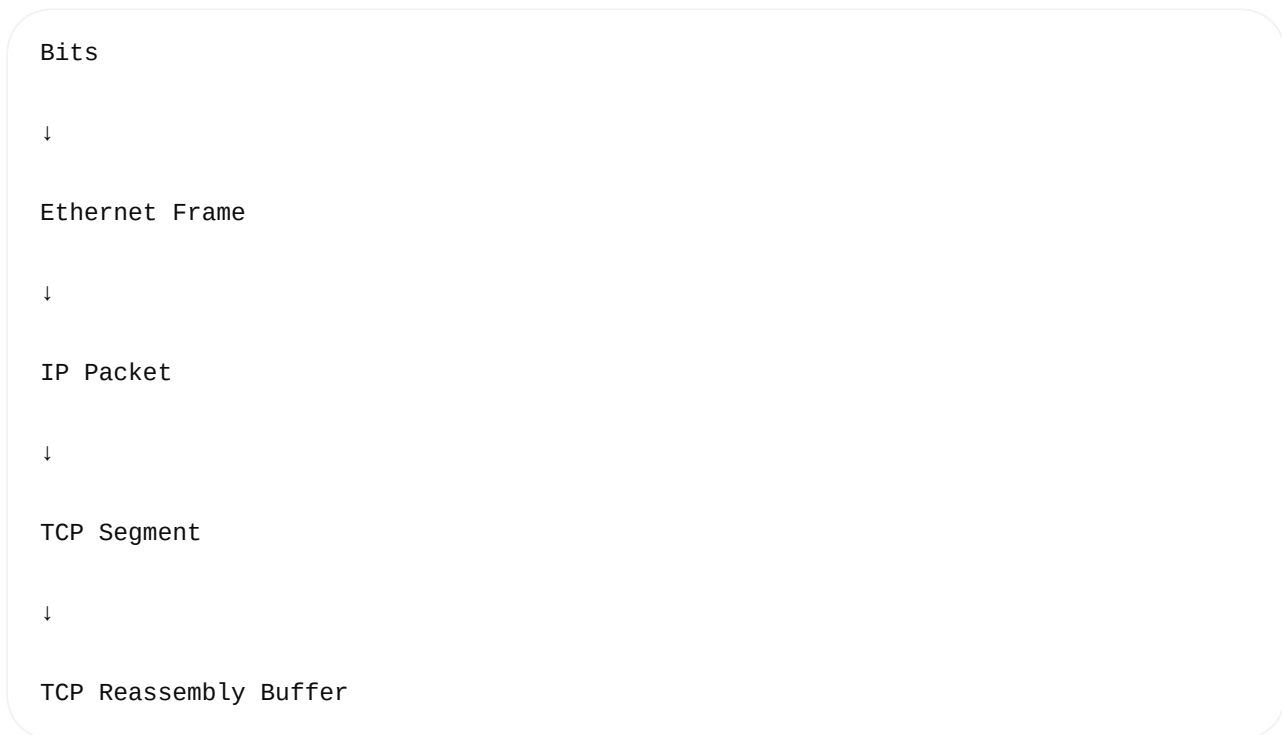
Remove IP Header

↓

TCP Segment



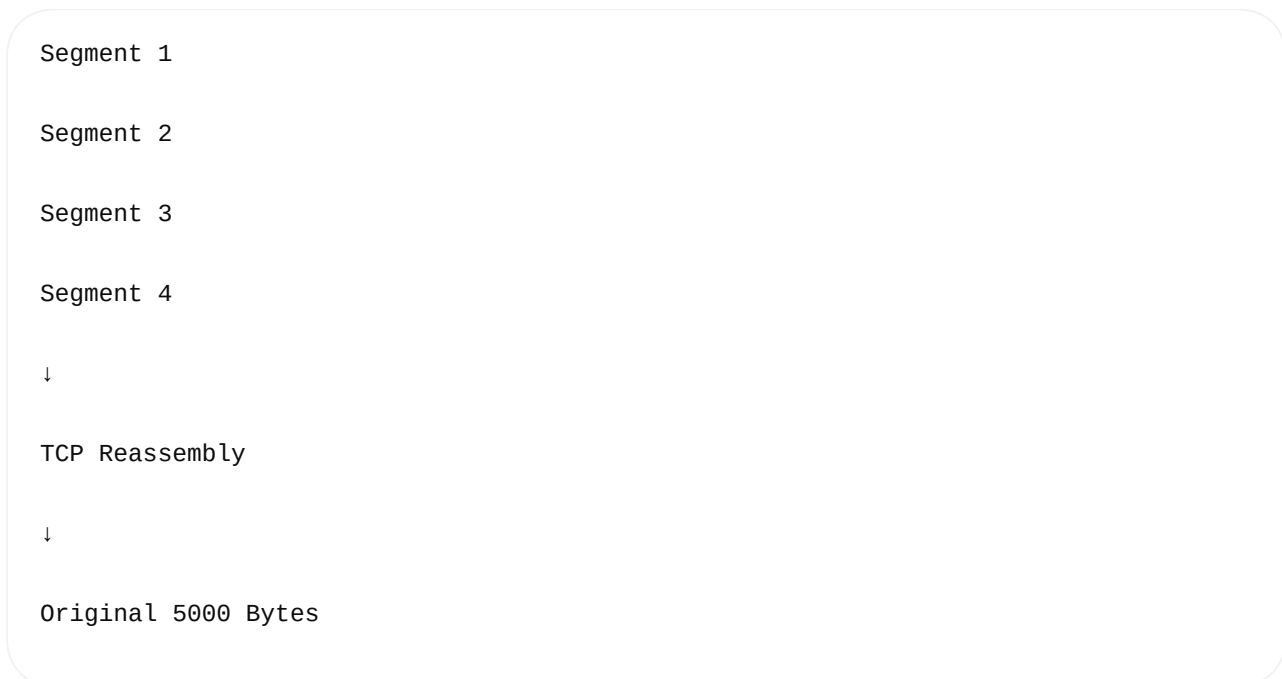
Frame 2



The same happens for Frames 3 and 4.

Step 7: TCP Reassembly

After all segments arrive:



The kernel verifies sequence numbers and reconstructs the original byte stream.

Only after successful reassembly does the kernel deliver the data to the application.

recv()

↓

Application receives

5000 Bytes

Complete Flow

Application

send(5000)

|

▼

TCP

Split into 4 Segments

|

▼

For EACH Segment

|

▼

IP Layer

Routing Lookup

Add IP Header

|

▼

Ethernet Layer

ARP Cache Lookup

Add Ethernet Header

|



NIC

Transmit

|



Switch

MAC Lookup

Forward

|



PC-B NIC

Receive Frame

|



Ethernet

Remove Ethernet Header

|



IP

Remove IP Header

|

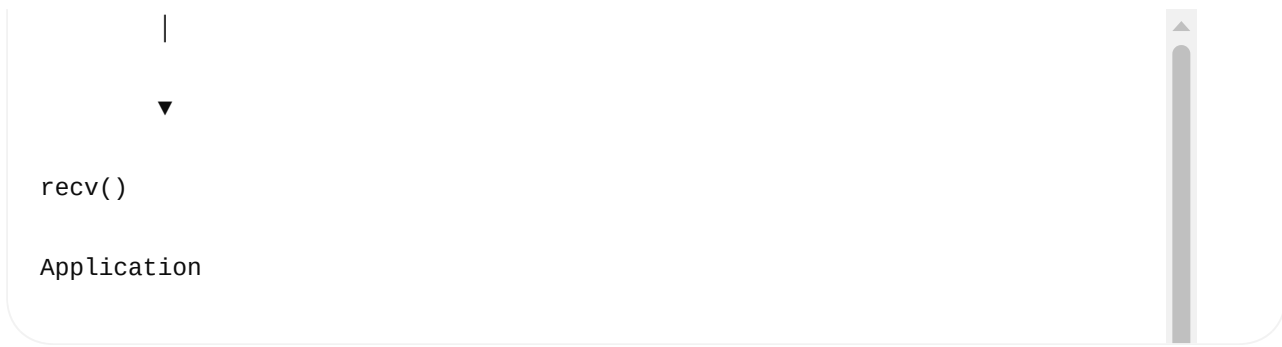


TCP

Collect Segments

Reassemble





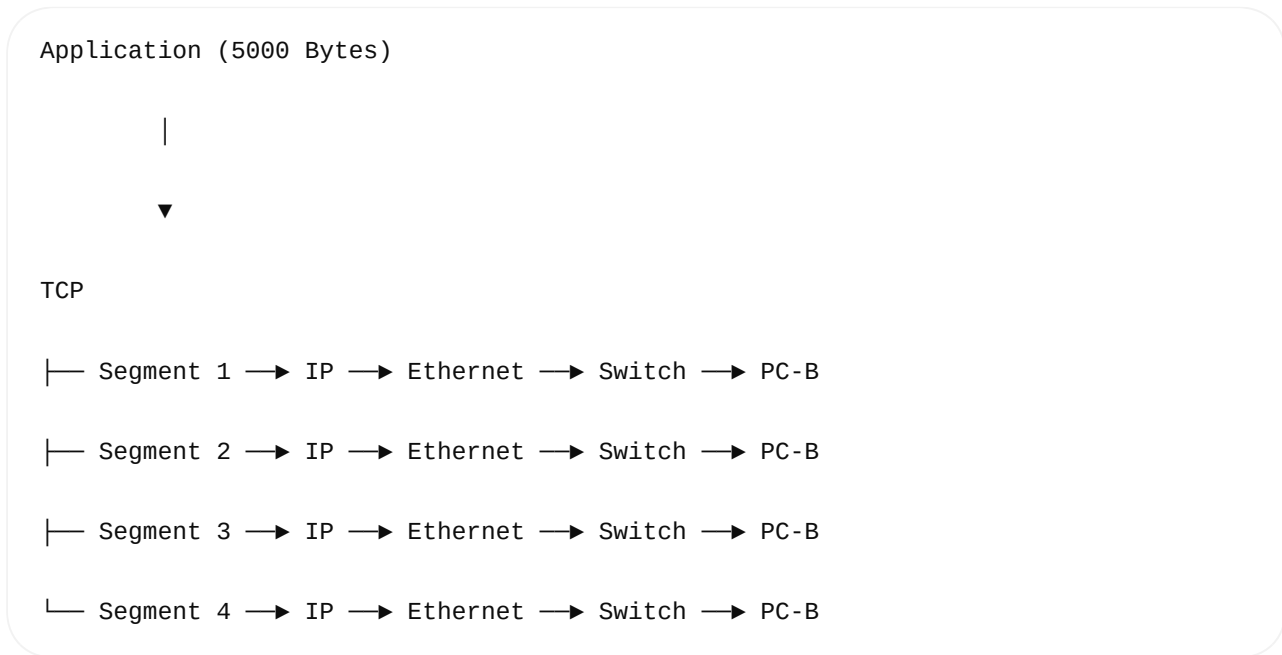
Key Interview Clarification

A common misconception is that:

"TCP splits the data once, and then the whole group of segments goes through IP and Ethernet together."

That's not how it works.

The actual processing is:



Each segment becomes its own IP packet and then its own Ethernet frame. Every frame is routed, encapsulated, forwarded by the switch, and received independently before TCP reassembles the original data on PC-B.

This **per-segment, per-packet, per-frame pipeline** is the mental model that networking engineers and Embedded Linux interviewers expect you to understand.